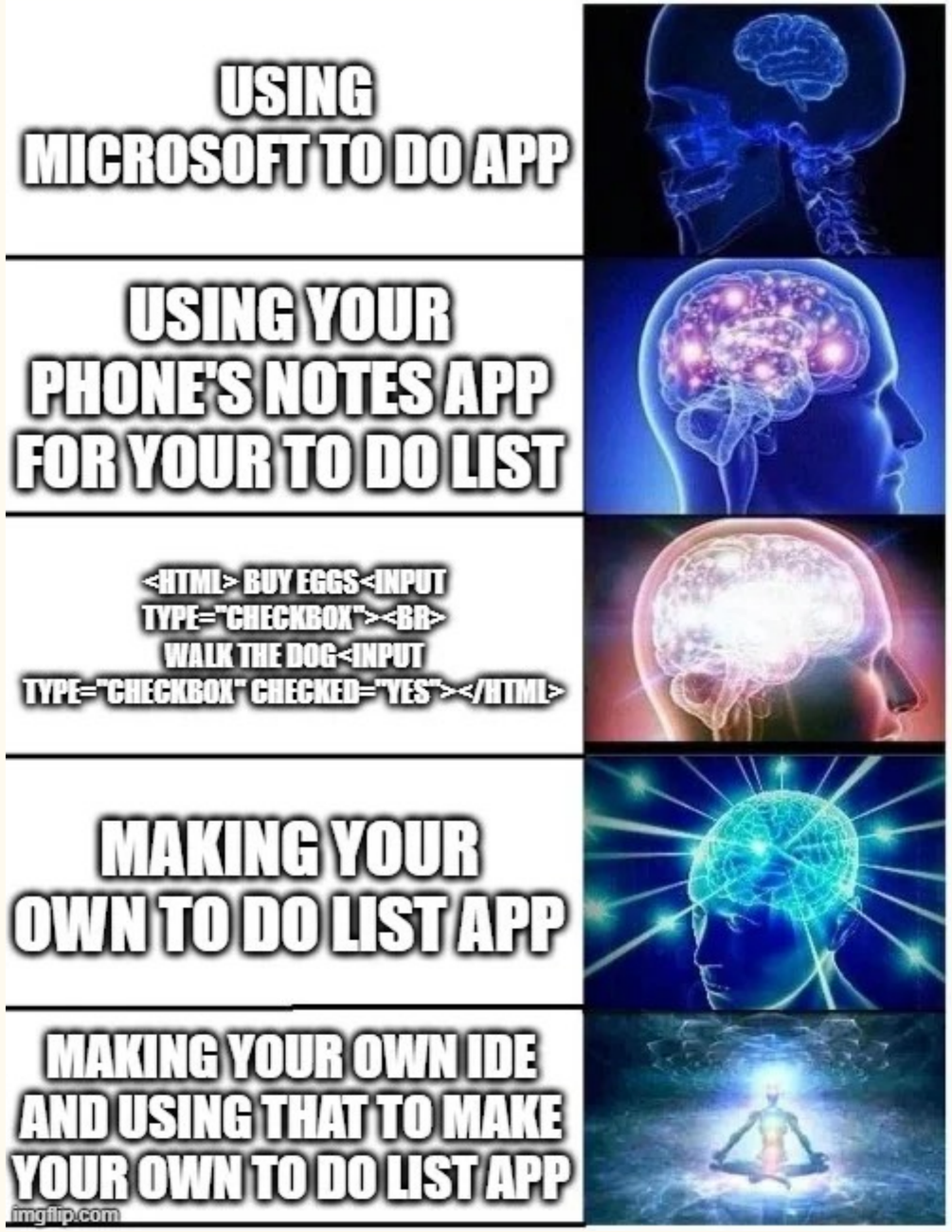
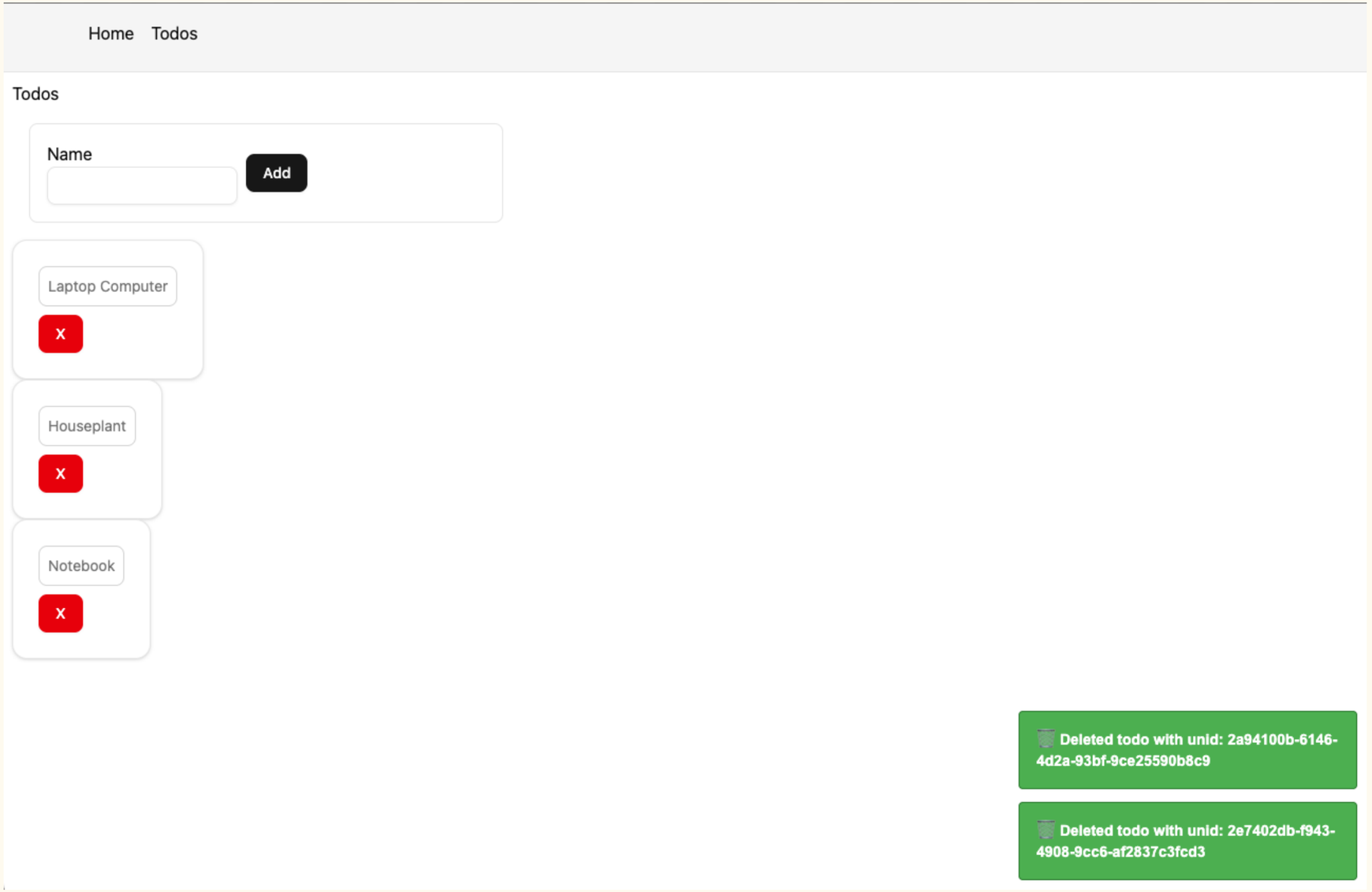
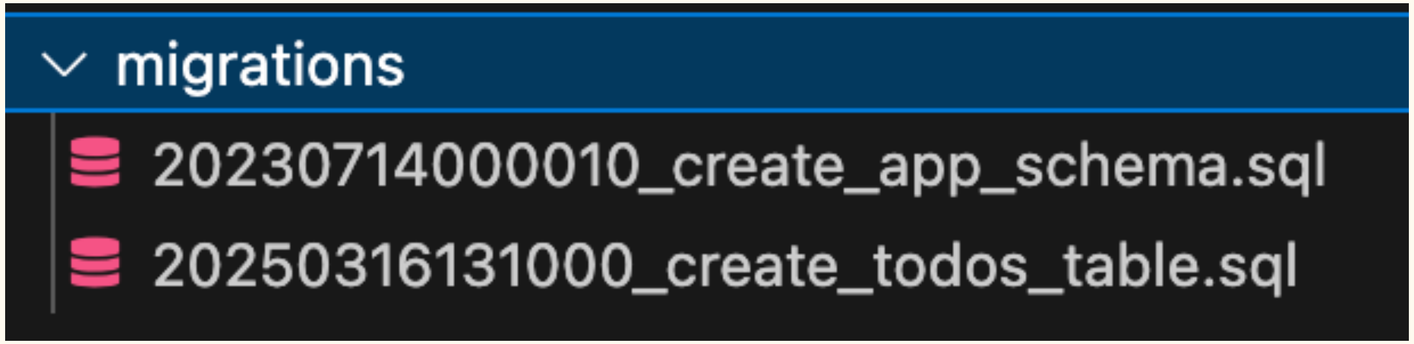
TODO APP


USING MICROSOFT TO DO APP
USING YOUR PHONE'S NOTES APP FOR YOUR TO DO LIST
<HTML> BUY EGGS<INPUT TYPE="CHECKBOX">
 WALK THE DOG<INPUT TYPE="CHECKBOX" CHECKED="YES"></HTML>
MAKING YOUR OWN TO DO LIST APP
MAKING YOUR OWN IDE AND USING THAT TO MAKE YOUR OWN TO DO LIST APP
imgflip.com

Project


Home
Todos
Todos
Name
Add
Laptop Computer
X
Houseplant
X
Notebook
X
Deleted todo with unid: 2a94100b-6146-4d2a-93bf-9ce25590b8c9
Deleted todo with unid: 2e7402db-f943-4908-9cc6-af2837c3fcd3

Migrations folder



```
CREATE SCHEMA IF NOT EXISTS w5_fullstack_todos;
```

```sql
CREATE TABLE IF NOT EXISTS todos (
    unid UUID PRIMARY KEY NOT NULL DEFAULT gen_random_uuid(),
    name CHARACTER VARYING(255) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Domain

```rust
// domain/todo/mod.rs
pub mod components;
pub mod page_todos;
pub mod pk;
#[cfg(feature = "ssr")]
pub mod todos_db;
pub mod todos_services;
```

build_app_router

```rust
pub async fn build_app_router(conf_file: ConfFile, pool: PgPool) -> anyhow::Result<Router> {
    let leptos_options = conf_file.leptos_options;

    let routes = generate_route_list(|| view! { <App /> });

    let app_state = AppState {
        leptos_options,
        pool: pool.clone(),
    };

    Ok(Router::new()
        .route(
            "/api/{*fn_name}",
            get(server_fn_handler).post(server_fn_handler),
        )
        // .layer(PropertyAccessLayer::new()) // custom middleware for properties
        .leptos_routes_with_handler(routes, get(leptos_routes_handler))
        .fallback(file_and_error_handler)
        .with_state(app_state))
}
```

CRUD

Read

```rust
pub async fn get_all(pool: &PgPool) -> Result<Vec<SqlThing>, sqlx::Error> {
    query_as!(
        SqlThing,
        r#"
            SELECT
                unid,
                name,
                created_at,
                updated_at
            FROM todos
        "#
    )
    .fetch_all(pool)
    .await
}
```

Create

```rust
pub async fn add(pool: &PgPool, name: String) -> Result<SqlThing, sqlx::Error> {
    query_as!(
        SqlThing,
        r#"
            INSERT INTO todos (name)
            VALUES ($1)
            RETURNING unid, name, created_at, updated_at
        "#,
        name
    )
    .fetch_one(pool)
    .await
}
```

Delete

```rust
pub async fn delete(pool: &PgPool, unid: Unid<TodosPk>) -> Result<Unid<TodosPk>, sqlx::Error>
{
    let deleted_todo = query_as!(
        SqlThing,
        r#"
            DELETE FROM todos
            WHERE unid = $1
            RETURNING unid, name, created_at, updated_at
        "#,
        unid.as_ref()
    )
    .fetch_one(pool)
    .await?;

    Ok(deleted_todo.unid)
}
```

ServerAction

- Type-safe bridge between client and server
- Handles form submissions and mutations
- Keeps sensitive logic on the server
- Automatically serializes/deserializes data via serde

```rust
let add_todo = ServerAction::<AddTodo>::new();
let delete_todo = ServerAction::<DeleteTodo>::new();

let todos = Resource::new(
    move || (add_todo.version().get(), delete_todo.version().get()),
    move |_| get_todos(),
);
```

Let’s code!

Your turn!